

You are a senior full-stack engineer on CyberGuard Pro.

Implement the Collaborators management and Targets system fully,
wired to the exact API spec below, consistent with the existing UI.

STEP 1 — Audit First

Read these files before writing anything:

- Project detail page/component (Team tab, any existing Targets tab)
- API service layer (existing collaborator/target functions if any)
- State management for current project
- Existing UI components (modals, tables, badges, confirm dialogs)
- How currentUser and project owner_id are accessed
- Router config for project sub-routes

SECTION A — Collaborators

Endpoints

- GET /api/projects/{project}/collaborators → list members
- PATCH /api/projects/{project}/collaborators/{user} → change role (owner only)
- DELETE /api/projects/{project}/collaborators/{user} → remove member (owner only)

Team Tab UI

- Load collaborators on tab open via GET
- Each row shows: avatar (or initials), name, email, role badge, actions
- Role badge colors: Owner → purple, Editor → cyan, Viewer → gray
- Owner row: show "Owner" badge, no actions (cannot change or remove self)

****Owner-only controls per member row:****

- Role dropdown (editor / viewer) → on change → PATCH immediately, no save button
 - On 200 → refresh list
 - On 403 → show "Only the owner can change roles"
- Remove button → confirmation popover → DELETE
 - On 200 → remove row from UI

- On 400 → show "You cannot remove yourself"
- On 403 → show "Only the owner can remove members"

****Hide all controls for non-owners proactively****

(check currentUser.id === project.owner_id before rendering)

SECTION B — Targets

Endpoints

- POST /api/projects/{project}/targets → add target
- GET /api/projects/{project}/targets → list targets
- GET /api/targets/{target} → single target detail
- PATCH /api/projects/{project}/targets/{target} → edit (owner/editor only)
- DELETE /api/projects/{project}/targets/{target} → delete (owner/editor only)

Request Body for POST and PATCH

```
```json
{ "type": "domain", "label": "Production API", "value": "example.com" }
```

**Type Rules (validate CLIENT-SIDE before submitting)**

type	value format	example
domain	hostname only	example.com
ip	IPv4 address	203.0.113.10
network	IPv4 CIDR	192.0.2.0/24

**Targets Tab UI (add as new tab inside project detail)**

text

Tabs: Overview | Team | Targets | Settings

**Targets list:**

- Table/card list: type badge, label, value, created date, actions
- Type badge colors: domain → cyan, ip → purple, network → orange
- "Add Target" button → visible to owner and editor only (hide for viewer)
- Edit/Delete actions → visible to owner and editor only

**Add/Edit Target Modal:**

- Type selector (domain / ip / network) — changes validation hint dynamically
- Label input (required)

- Value input with dynamic placeholder:
  - domain → "e.g. example.com (no https://)"
  - ip → "e.g. 203.0.113.10"
  - network → "e.g. 192.0.2.0/24"
- Client-side validation before submit:
  - domain: no http/https, valid hostname format
  - ip: valid IPv4
  - network: valid CIDR notation
- On 201 → close modal, append to list
- On 422 → show inline field errors
- On 403 → show "Viewers cannot add targets"

**Delete target:**

- Confirmation dialog
- On 200 → remove from list
- On 403/404 → show error message

**Single target detail** (optional drawer or page):

- GET /api/targets/{target} on row click
- Show all fields

**ROLE PERMISSION MATRIX**

Action	Owner	Editor	Viewer
View collaborators	✓	✓	✓
Change member role	✓	✗	✗
Remove member	✓	✗	✗
View targets	✓	✓	✓
Add target	✓	✓	✗
Edit target	✓	✓	✗
Delete target	✓	✓	✗

Enforce this matrix in the UI proactively.  
Determine current user role from project.collaborators or owner\_id comparison.

**ERROR HANDLING**

Status	Action
200/201	Success toast + update local state
400	Show specific API message

Status	Action
403	Hide UI element + show inline "permission denied" note
404	Show "not found" state
422	Map errors to specific form fields

---

## RULES

- Bearer token on every request (via existing interceptor)
- No hardcoded values — everything from API response
- Loading states on all async operations
- After every mutation → update local state (no full page reload)
- All new UI matches existing dark cyber theme exactly
- Use existing modal, toast, badge, confirm dialog components

---

## DELIVERABLES

1. All new/modified files list
2. Complete working code for each file
3. Role permission matrix enforced and verified
4. All endpoints wired and tested against these scenarios:
  - Owner can change roles and remove members
  - Non-owner cannot see role/remove controls
  - Owner/editor can add, edit, delete targets
  - Viewer sees targets but has no add/edit/delete controls
  - All error states handled gracefully